howard@cpu2006:~/exp/llvm-skil-test$ ./evaluate --skill llvm-optimization

llvm-optimization skill × CPU2006 13 項

測試 · 效能分析 · 優化

+3.3%
逐項調校 vs `default<O3>`

+23.9%
vs 原研究 full pipeline

2434
配對計時樣本

0
採用設定的輸出差異

skill：gmh5225/awesome-llvm-security · .claude/skills/llvm-optimization @ b7609b4 · 2026-11-13 · 13 項幾何平均，確認階段 12 對

01 實驗設計：三階段，避免挑選偏誤

raw.bc
原研究 13 項共用 bitcode
LLVM 14 (C/C++) · 19 (Fortran)

→

stage 1
skill 直接建議
O2 · O3 · Os · LTO · 目標 ISA

→

remarks
`-pass-remarks-output`
找出要調整的 pass

→

stage 2
inline / unroll / SLP knob
只用來挑設定

→

stage 3
全新 12 輪獨立確認
報告數字全部出自這裡

固定條件

> 同一 raw.bc、`llc -O=2`、同連結參數
> 原研究的固定輸入與工作目錄
> i7-14700K，固定 CPU 2

計時

> 每輪所有變體隨機排序各跑一次
> 同輪配對比值 → 幾何平均
> 2434 樣本（770 + 1040 + 624），全 exit 0

正確性

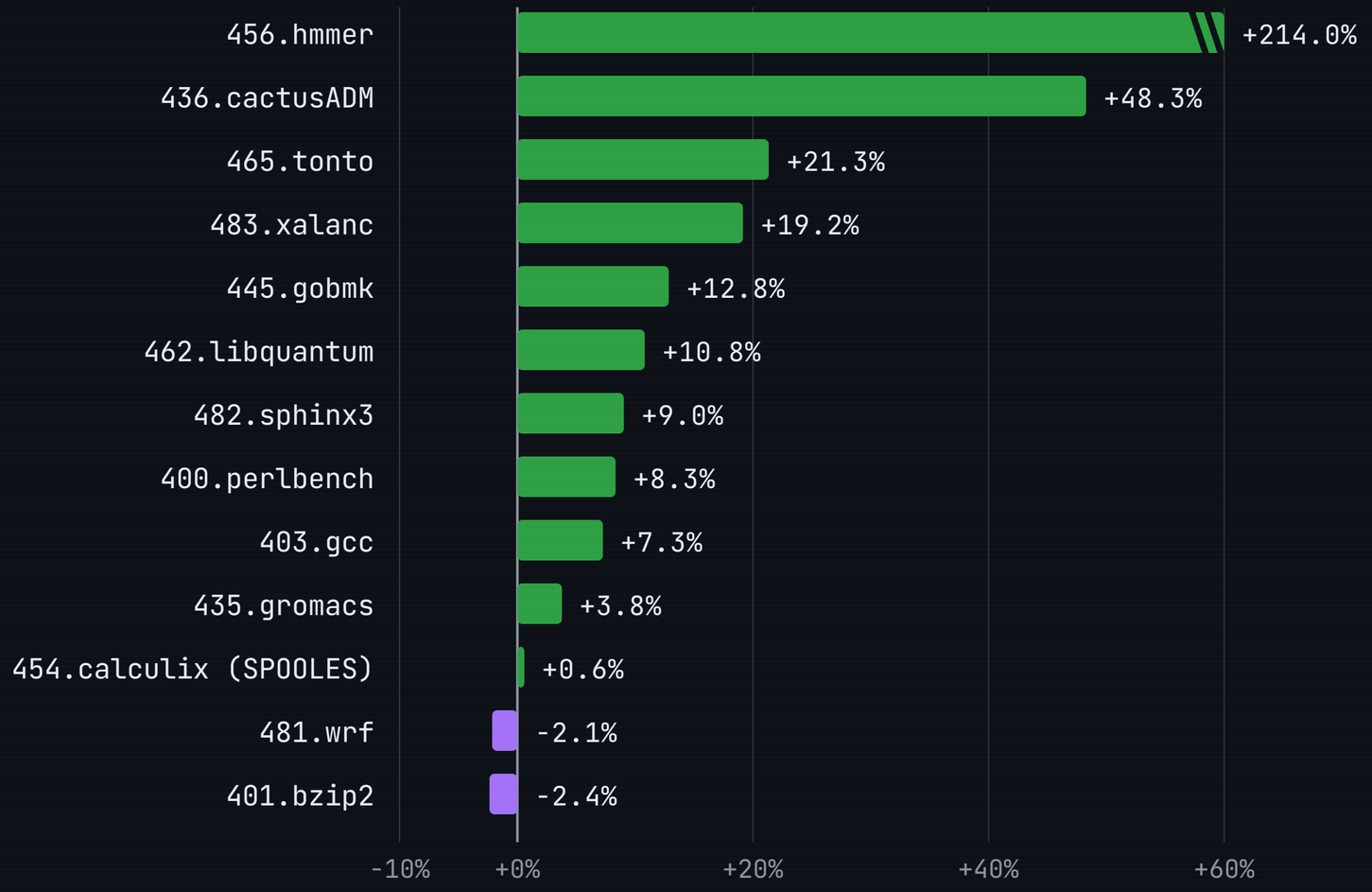
> 每個變體與已驗證的 `full` 差分比對
> 149 MATCH · 6 DIFF（皆為 FMA，見 06）
> 採用設定 100% 逐位元組相同

02 skill 內容如何變成可量測的變體

skill 段落	變體	實際指令
Optimization Levels	`o2` `o3` `os`	`opt -passes='default<O2|O3|Os>'`
Link-Time Optimization	`lto`	`lto-pre-link<O3>,internalize,lto<O3>` • 只保留 `main,_QQmain`
Vectorization（目標 ISA）	`*_native`	IR `"target-cpu"` → `alderlake`（14700K 同 ISA）
Optimization Remarks	分析	`-pass-remarks-output -pass-remarks-filter='inline|loop-vectorize|slp|licm|unroll|gvn'`
Inlining • Unrolling • SLP	stage-2 knob	`-inline-threshold=500` • `-unroll-threshold=600` • `-slp-threshold=1000`

對照組：`full`＝原研究 15-pass 顯式 pipeline、`o3`＝`default<O3>`，直接複製原研究已驗證的執行檔。skill 中「撰寫自訂 pass / dominator tree / Alive2」屬於 pass 開發，不適用於調整既有程式，未納入。

03 第一個發現：O3 比研究用 pipeline 快 +19.9%



讀法

每列為 `default<O3>` 相對原研究 `full` 的速度變化，確認階段 12 對幾何平均。

重點

> HMMER +214.0%（約 3.1 倍；圖中截斷）
> Cactus +48.3%、Tonto +21.3%、Xalan +19.2%
> 只有 bzip2（-2.4%）與 WRF（-2.1%）是 `full` 較快

skill 其他等級

> `-O2` 與 O3 差距多在 ±3% 內
> `-Os` .text −4% 至 −31%，但 Xalan -9.3%、Cactus -13.1%

04 Remarks 指出差距來源

Passed remarks	full	O3	LTO+ISA
GVN · gcc	0	9,725	10,966
GVN · Xalan	0	4,860	7,095
loop-vectorize · Tonto	3,541	450	**2,208**
inline · Xalan	54,623	60,456	**66,719**
inline · gcc	17,889	20,899	**21,554**

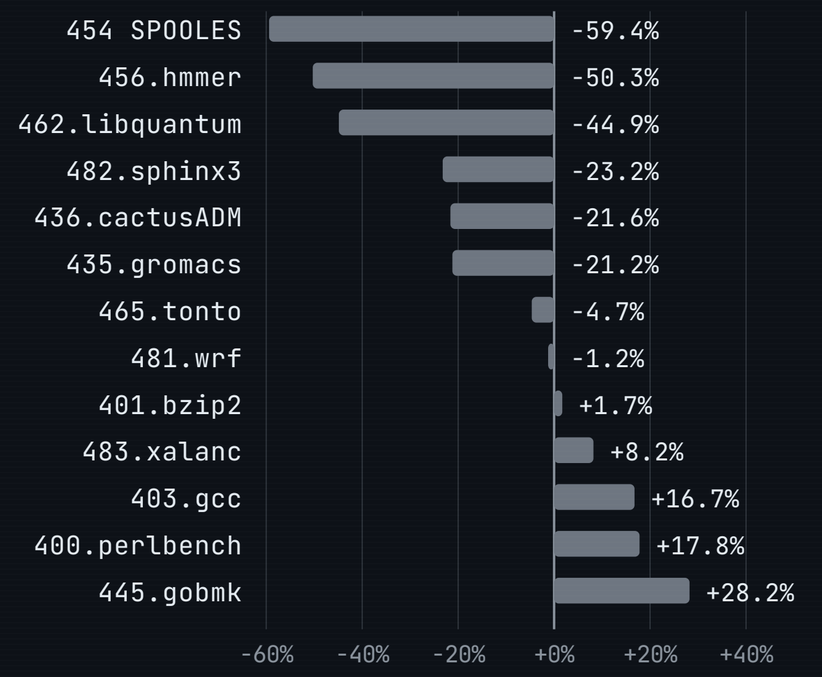
> **原研究 pipeline 完全沒有 GVN**：O3 有數千筆冗餘消除，與 O3 整體快 20% 一致。

> **Tonto 向量化**：x86-64 (SSE2) 成本模型讓多數 Fortran 迴圈不值得向量化；改 AVX2 後成功數增加約 5 倍。

> **LTO 每項都 inline 更多**，且刪除未被呼叫的函式庫程式碼（右圖）。

remark 計數是嘗試次數，不等於速度；只用來挑 stage-2 要調的 pass。

LTO + inline 500：.text 相對 O3



05 stage 1：skill 直接建議的設定（相對 O3）

LTO

+5.4%

Perl；gobmk +2.4%、Xalan +1.7%

Cactus -7.6%：LTO 反而較慢

目標 ISA（Alder Lake）

+11.2%

WRF LTO+ISA；Sphinx O3+ISA +4.7%

C/C++ 原本已是 x86-64-v3，增益小

-O2 / -Os

≈ O3

`-O2` 多在 ±3% 內

`-Os` Xalan -9.3%、Cactus -13.1%

libquantum 在本階段的 O3 樣本有一次主機噪音尖峰（943 ms vs 常態約 390 ms），其 stage-1 比值不採用；stage 2/3 的 10–12 對中沒有變體勝過 O3。

06 目標 ISA 的陷阱：Fortran + FMA 改變輸出

```
# Flang 前端產生的 IR（WRF）
attributes #0 = { "frame-pointer"="all" "target-cpu"="x86-64" }
  11431  fmul contract float
   5414  fadd contract float

# target-cpu → alderlake：contract 被融合成 FMA
DIFF  Cactus · Tonto · WRF   (6 個 native 變體)
  WRF  em_t_2  46359.5703 → 46359.5664   # 1 ULP (float32)
  Tonto Kinetic Energy 76.771588 → 76.771596

# 修正：保留 AVX2，關閉 FMA
"target-cpu"="alderlake" "target-features"="-fma"
MATCH 6/6 — 逐位元組相同
```

結果（stage 1，相對 O3）

	含 FMA	無 FMA
WRF LTO+ISA	+11.2%	+9.7%
Tonto O3+ISA	+3.6%	-0.5%
Cactus O3+ISA	-4.7%	-3.8%

解讀

> WRF 的增益來自 AVX2 寬度，關 FMA 仍保留

> Tonto 的增益全來自 FMA；不能逐位元組相同就不採用

> skill 未提醒此風險

07 stage 2：在嚴格正確的 LTO+ISA 上調整 knob

相對 O3（10 對）	O3+ISA	LTO+ISA	+inline 500	+SLP off	+unroll 600
400.perlbench	+3.1%	+3.9%	**+8.4%**	+6.8%	+5.1%
401.bzip2	-0.9%	-0.6%	+0.2%	-1.2%	-0.7%
403.gcc	+0.3%	-0.1%	**+2.2%**	+0.5%	+0.9%
435.gromacs	+1.1%	+0.6%	+0.3%	-3.1%	+0.7%
436.cactusADM	-3.1%	-7.2%	-0.3%	-0.6%	-6.9%
445.gobmk	+0.9%	+2.1%	**+3.2%**	+2.2%	+3.1%
454.calculix (SPOOLES)	-0.1%	+1.2%	+1.8%	**+15.5%**	+1.0%
456.hmmer	+1.6%	+0.9%	+2.2%	+0.8%	**+2.5%**
462.libquantum	-0.0%	-3.7%	-0.3%	-0.6%	-2.4%
465.tonto	-0.3%	-1.7%	+0.8%	-1.6%	-1.5%
481.wrf	+9.5%	+10.8%	+9.8%	+9.9%	**+11.3%**
482.sphinx3	**+2.2%**	+2.1%	+1.6%	-4.7%	+1.5%
483.xalanc	-0.5%	+0.7%	**+3.9%**	+0.7%	+0.0%

突出結果

> **SPOOLES 關 SLP +15.5%**：與原研究消融中 SLP 造成 −11% 一致

> **inline 500** 是最一致的 knob：13 項幾何平均 +2.5%

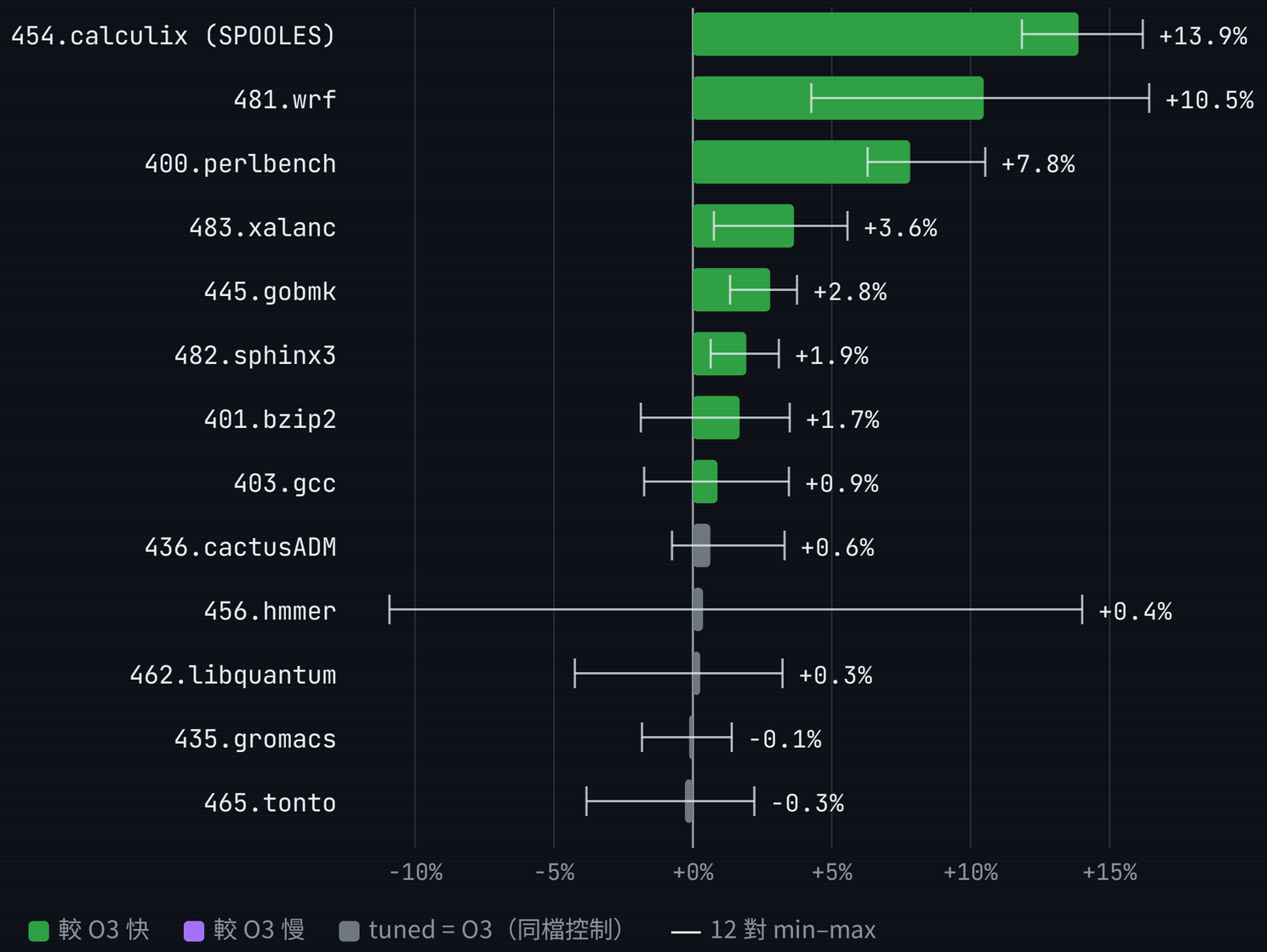
> WRF 各欄都約 +10%：來源是 ISA，不是 knob

> Cactus：LTO 系列 −7%，維持 O3

噪音尺度

5 項 `-unroll-threshold=600` 產生與 LTO+ISA 完全相同的檔案，兩欄差距（約 ±1.5%）即為噪音。|變化| < 1.5% 以灰底表示。

08 stage 3：獨立 12 輪確認，13 項 +3.3%



重現 stage 2

> SPOOLES +15.5% → **+13.9%**
> WRF +11.3% → **+10.5%**
> Perl +8.4% → **+7.8%**
> Xalan +3.9% → **+3.6%**
> gcc +2.2% → +0.9%（不足以宣稱）

控制組

5 項 tuned 即 O3 本身，同檔配對落在 -0.29% 至 +0.63%。統一設定 `lto_inl` 為 +2.1%。

09 建議設定（只在明顯超過噪音時離開 O3）

工作量	建議設定	vs O3	vs full
400.perlbench	LTO · inline 500 · SLP off	+7.8%	+16.8%
401.bzip2	原研究 full pipeline	+1.7%	-0.7%
403.gcc	LTO · inline 500	+0.9%	+8.2%
435.gromacs	default<O3>	-0.1%	+3.6%
436.cactusADM	default<O3>	+0.6%	+49.2%
445.gobmk	LTO · inline 500 · unroll 600	+2.8%	+16.0%
454.calculix (SPOOLES)	LTO · inline 500 · SLP off	+13.9%	+14.6%
456.hmmer	default<O3>	+0.4%	+215.1%
462.libquantum	default<O3>	+0.3%	+11.1%
465.tonto	default<O3>	-0.3%	+21.0%
481.wrf	LTO · ISA(無FMA) · unroll 600	+10.5%	+8.1%
482.sphinx3	O3 · Alder Lake ISA	+1.9%	+11.1%
483.xalanc	LTO · inline 500	+3.6%	+23.5%

通用起點

```
opt -passes='lto-pre-link<O3>,
  internalize,lto<O3>' \
  -internalize-public-api-list=main \
  -inline-threshold=500 in.bc
# Fortran：target-cpu=alderlake
#          target-features=-fma
```

13 項 +2.1% vs O3

逐項例外

> 數值核心（SPOOLES）先試關 SLP

> Cactus 不用 LTO

> bzip2 原 pipeline 比 O3 快

10 skill 評估

有用

- 最佳化等級、LTO、目標 ISA 三個方向找出全部主要增益
- Optimization Remarks 直接指出 GVN 缺席、Tonto 向量化與 LTO inlining
- Inlining／SLP 的概念說明對選 knob 有幫助

不足或與工具鏈不符

- `opt -instcombine in.ll`（Alive2 範例）在 LLVM 19 失敗，需 `-passes=`
- `opt -stats` 在兩個 Debian release 工具鏈無輸出
- 未說明 new PM 下如何關 SLP：`-vectorize-slp=false` 無效，需 `-slp-threshold`
- 未提醒改 ISA 會讓 `contract` 浮點融合成 FMA
- 沒有 PGO、ThinLTO 實際流程；大段內容偏向 pass 開發

11 限制與下一步

限制

- 共用主機、未關 turbo；同檔控制單對仍可偏差 10% 以上，2% 以下不作結論
- repo 內程式與固定輸入，不是官方 SPEC 分數；454.calculix 實為 SPOOLES 元件
- 只在 i7-14700K 驗證；alderlake ISA 執行檔需要 AVX2
- tuned 在同一輸入上選出，已獨立重測，換輸入或機器仍需重驗

下一步

- PGO（instrumented / sample-based）與 ThinLTO
- 把 GVN 加入研究用 pipeline，重做消融以分離其貢獻
- 以 Flang `-O2/-O3` 前端重產 Fortran IR，比較 FIR 層最佳化
- 換輸入（SPEC train/ref）與另一台機器驗證 tuned 設定